Informe Final – Algoritmos y Estructuras de Datos II

Integrantes: Fernandez Cristian, Herrera Ailen, March Graciana, Simionato Lucas

Pregunta 1: ¿Cómo fue modelado el Warehouse?

El warehouse se modeló como un rectángulo sobre una grilla de celdas unitarias, con origen en (0, 0) y dimensiones (ancho_x, largo_y). Cada elemento físico es un rectángulo con ubicación, ancho y alto: racks, zonas de despacho y pasillos. Las puertas son celdas sobre el borde y los pickers son puntos con posición, capacidad e historial. Los productos se guardan dentro de los racks en una terna (x, y, z), donde z es la altura.

La clase Warehouse concentra el estado y valida cada elemento al agregarlo. No es un contenedor pasivo: controla que los racks tengan un lado igual a 2, que las puertas estén sobre el borde, que cada puerta tenga una zona de despacho delante, que no haya solapamientos y que los pasillos no se pisen. Además, mantiene índices de celdas ocupadas para que las validaciones sean O(1) por celda. La representación física se usa para validar y dibujar; para navegar se construye un grafo aparte.

Pregunta 2: ¿Qué representación de la estructura de datos utilizaron y por qué?

Usamos dos representaciones complementarias. Para el espacio físico, una grilla implícita: no guardamos una matriz de celdas, sino que cada elemento sabe qué celdas ocupa y el Warehouse mantiene diccionarios de celdas bloqueadas, celdas de pasillo y posiciones de pickers. Para la navegación, un grafo dirigido con lista de adyacencia implementada como diccionario de diccionarios: `_adj[u] = {v: peso}`.

Elegimos la lista de adyacencia porque el grafo es disperso: cada celda tiene a lo sumo cuatro vecinas, así que la cantidad de aristas es O(V), no $O(V^2)$. Una matriz de adyacencia para el escenario grande tendría 1.898 × 1.898 entradas, casi todas vacías; la lista guarda solo 5.963 aristas. Además, recorrer los vecinos de un nodo es O(grado), que es exactamente lo que necesitan BFS y A*. El diccionario exterior da acceso O(1) por celda y el interior permite consultar una arista y su peso en O(1).

También usamos estructuras auxiliares: `deque` para la cola de BFS, `heapq` para la cola de prioridad de A*, diccionarios para la tabla de distancias y listas ordenadas para la cola de waves. Todas son estructuras de la biblioteca estándar de Python, no de terceros, y se explican en el código.

Pregunta 3: ¿Qué algoritmos utilizaron para resolver los problemas encontrados?

Para construir el grafo usamos un recorrido de la grilla y la regla de circulación de la consigna. Para validar alcanzabilidad, dos BFS: uno en el grafo normal y otro en el invertido. Para calcular caminos mínimos, BFS y A* con heurística Manhattan. Para medir la distancia de una wave o de las zonas de despacho a todas las celdas, BFS multi-origen. Para detectar la circulación viva, componentes fuertemente conexas calculadas con recorridos en el grafo normal y en el invertido.

Para armar waves usamos tres estrategias greedy: por pedido, por cercanía con prioridad y solo por cercanía. Para asignar pickers, una cola por prioridad y una elección greedy del picker más cercano que

pueda cargar la wave, con best fit como desempate. Para ordenar las paradas, dos estrategias: orden de los pedidos y vecino más cercano, con una regla adicional para no dejar paradas atrás en pasillos de un solo sentido. Para la simulación, un bucle por ticks con resolución de colisiones, esperas, retrocesos, estacionamiento y desvíos con BFS.

Pregunta 4: ¿Por qué eligieron dichos algoritmos?

Porque el problema es, en el fondo, un problema de caminos mínimos en un grafo no ponderado y dirigido. BFS resuelve ese caso de forma exacta y con complejidad O(V + E), que es óptima para recorrer el grafo. A* resuelve el mismo problema con una heurística admisible y permite comparar el efecto de la información adicional sobre la cantidad de nodos expandidos. Los dos son algoritmos de la materia y se pueden explicar y justificar sin cajas negras.

Para las waves y el orden de paradas elegimos greedy porque la consigna no pide el óptimo global y prohíbe metaheurísticas y solvers. Un greedy determinista es fácil de reproducir, de medir y de comparar; además, permite aislar el efecto de una decisión (prioridad, distancia, capacidad) y entender por qué un escenario mejora o empeora. La alternativa exacta —probar todas las particiones y todos los órdenes— tendría un costo factorial inviable para 30 pedidos y 14 waves.

También elegimos estas herramientas porque el código tenía que quedar claro para la defensa oral. Preferimos un algoritmo simple que podamos explicar línea por línea antes que una optimización difícil de justificar. La comparación experimental muestra que, aun siendo simples, las estrategias logran mejoras importantes.

Pregunta 5: ¿Qué ocurre si todos los pasillos tienen el mismo largo?

El algoritmo no depende de que los pasillos tengan largos distintos. El grafo se construye celda por celda y la única información que importa es qué celdas son transitables y en qué dirección. Si todos los pasillos tienen el mismo largo, el warehouse tiende a ser más regular y las distancias dependen sobre todo de la cantidad de pasillos y de la posición relativa de los racks, no de la longitud individual. En un layout regular, BFS explora “en rombo” y A* con Manhattan se orienta muy bien; los empates de distancia son más frecuentes, pero se resuelven de forma determinista por orden de inserción y por ID.

Si los pasillos son todos del mismo largo y además todos AMBOS, el problema se parece a una grilla libre y la distancia entre dos celdas se acerca a la Manhattan. Si son de un solo sentido, la dirección pesa más que el largo: un pasillo corto en contra del sentido puede obligar a un rodeo mucho más largo que un pasillo largo a favor. En nuestros escenarios hay pasillos de largos distintos, pero el escenario básico y el de cercanía tienen layouts bastante regulares y se comportan como describe esta explicación.

No hicimos un experimento específico que aísle la variable “todos los pasillos iguales”; lo que sí hicimos fue comparar layouts regulares y con esquinas. Si el equipo quiere agregar una medición puntual, se puede generar un escenario con todos los pasillos de la misma longitud y correr las mismas cuatro combinaciones. La conclusión esperada es que no cambia la correctitud del algoritmo, solo la forma de los recorridos y la frecuencia de los empates.

Pregunta 6: ¿Cómo cambia el comportamiento del algoritmo cuando aumenta el tamaño del Warehouse?

El grafo crece con la cantidad de celdas transitables, y el costo de BFS y A* crece con V + E. En la práctica, el crecimiento es aproximadamente lineal en la cantidad de celdas, con constantes que dependen de la

cantidad de vecinos. En los escenarios generados, pasar de 177 a 355 nodos hizo subir el tiempo medio de ruta de 0,23 a 0,74 ms; el tiempo de armado de waves subió de 12,5 a 32,5 ms. En el escenario grande, con 1.857 nodos, el tiempo medio por wave con cercanía + BFS fue 2,37 ms, y el armado de las 14 waves, 228 ms.

El crecimiento no es solo cuestión de nodos: también influyen la cantidad de paradas por wave, la cantidad de waves, la cantidad de pickers y la forma del layout. Un warehouse más grande pero con pasillos anchos y buen conectividad puede ser más fácil de recorrer que uno chico con muchos callejones. La tabla de distancias memoizada amortigua el crecimiento del armado: se corre un BFS por rack distinto, no por ítem. En el escenario grande, 68 BFS alcanzan para 14 waves.

El costo de construir el grafo de navegación también crece: 11,15 ms para el grafo completo y 34,98 ms para el de navegación en el escenario grande. Es un costo que se paga una vez por escenario. Si el warehouse creciera a decenas de miles de celdas, habría que pensar en construcciones incrementales o en precomputar distancias, porque la tabla actual guarda una fila por rack y puede ocupar bastante memoria.

Pregunta 7: ¿Cómo determinaron la distancia entre las ubicaciones de una wave?

La distancia entre dos ubicaciones se calcula como el camino mínimo en el grafo dirigido, con BFS o A*. Cada arista cuesta 1, así que la distancia es la cantidad de pasos del picker. Para un producto, la ubicación no es la celda del rack sino cualquiera de las celdas de pasillo ortogonalmente adyacentes desde las que se puede recoger. Por eso la distancia a un producto es la menor distancia a su conjunto de celdas de recogida.

Para armar waves no alcanza con la distancia entre dos productos: hace falta la distancia de la wave completa a cada candidato. La solución es una tabla de distancias por rack. La primera vez que se necesita un rack, se corre un BFS multi-origen desde sus celdas de recogida y se guarda la fila {celda: distancia}. La distancia de la wave a una celda es el mínimo entre las filas de sus racks. Al agregar un ítem, se actualiza ese mínimo celda por celda. Así se obtiene lo mismo que un BFS multi-origen desde todas las celdas de recogida de la wave, pero sin recalcular los racks que ya están en la tabla.

En el ruteo, la distancia entre paradas se calcula con `bfs_to_any` o `astar_to_any`, que buscan el camino mínimo hasta cualquiera de las celdas de recogida de la parada siguiente. En la estrategia de cercanía se hace una sola búsqueda hacia todas las paradas pendientes, y el primer destino que se toca es el más cercano. La altura z no se incluye en la distancia de camino; se suma aparte como costo de recogida, porque no cambia el recorrido sino el tiempo que el picker está quieto.

Pregunta 8: ¿Cómo decidieron qué pedidos pertenecen a una misma wave?

Con una estrategia greedy determinista que respeta las restricciones de capacidad y prioridad. Hay tres variantes. En `por_pedido`, se ordenan los pedidos por prioridad y se abre una wave con el más prioritario; después se agregan pedidos completos mientras entren, eligiendo por prioridad y, si empatan, por cercanía a la wave. En `por_cercania`, se trabaja ítem por ítem: la semilla es el ítem del pedido más prioritario y se agrega el ítem de mayor prioridad que esté más cerca, aunque sea de otro pedido. En `solo_cercania`, se ignora la prioridad y se agrega siempre el ítem más cercano.

Las tres respetan el máximo de pedidos distintos, el máximo de unidades y el peso máximo de la wave. El peso máximo se toma como la capacidad del picker operativo más grande, para que siempre haya alguien que pueda cargarla. Si un pedido no entra completo en una wave vacía, se divide: se agregan los ítems que

caben y el resto queda para la siguiente. En las estrategias por ítem la división es más frecuente, pero permite agrupar productos cercanos de pedidos distintos.

La estrategia que quedó como opción por defecto en la aplicación es `por_cercania`, porque respeta la prioridad y además aprovecha la cercanía. `solo_cercania` obtiene mejor distancia en el escenario grande (2.167 contra 2.481), pero puede postergar pedidos urgentes; por eso se ofrece como alternativa de comparación. `por_pedido` es la más conservadora y la que menos divide pedidos, y sirve como línea base.

Pregunta 9: ¿Qué estrategia utilizaron para ordenar las ubicaciones de una wave?

Implementamos dos. La primera, `orden`, visita las paradas en el orden en que aparecen los pedidos dentro de la wave. La segunda, `cercania`, es un greedy de vecino más cercano: en cada paso elige la parada pendiente más cercana a la celda actual. La cercanía no hace una búsqueda por parada: hace un solo BFS hacia todas las celdas de recogida pendientes y corta en la primera que toca, que es la más cercana. Si varias paradas comparten la celda de llegada, se sirven todas juntas.

La cercanía tiene una regla adicional para los pasillos de un solo sentido: antes de elegir el próximo destino, descarta las celdas de recogida que obligarían a dejar atrás una parada pendiente. Para eso usa el progreso de la celda dentro del pasillo (y en N, −y en S, x en E, −x en O, 0 en AMBOS). Si una parada pendiente solo se puede recoger desde celdas con menor progreso en el mismo pasillo, la celda actual queda bloqueada y se sirve primero la otra. Esa regla evita rodeos largos y fue uno de los últimos ajustes del proyecto.

Las dos estrategias usan BFS o A* para cada tramo. La diferencia está en qué parada se elige, no en cómo se calcula el camino. Las métricas de cada recorrido permiten comparar ambas sobre la misma wave y con el mismo buscador.

Pregunta 10: ¿Qué diferencias existen entre las estrategias implementadas?

Hay diferencias en tres niveles. En el ruteo, `orden` conserva la secuencia de los pedidos y `cercania` reordena las paradas por distancia. En la generación de waves, `por_pedido` agrupa pedidos completos, `por_cercania` agrupa ítems de distintos pedidos respetando prioridad y `solo_cercania` agrupa ítems sin mirar prioridad. En el buscador, BFS y A* encuentran la misma distancia mínima, pero A* expande menos nodos y suele tardar más en tiempo de pared por el costo del heap.

La diferencia práctica más visible es la distancia. En el escenario grande, `cercania` recorre 2.481 pasos contra 3.751 de `orden` (−33,9 %). En el escenario de racks en esquinas, en cambio, `orden` recorre 26 contra 28 de `cercania`: el greedy se equivoca porque la parada “más cercana” en línea recta obliga a un rodeo por la geometría. Ese contraste es el mejor ejemplo de que las estrategias no son intercambiables.

En waves, `solo_cercania` obtiene 2.167 pasos, `por_cercania` 2.481 y `por_pedido` 2.904. La diferencia entre las dos primeras es el respeto por la prioridad; la diferencia entre `por_cercania` y `por_pedido` es que la primera puede dividir pedidos y agrupar ítems cercanos de pedidos distintos. En tiempo de armado las tres están cerca (222,8 a 235,5 ms), pero el ruteo es más barato cuando las waves quedan mejor agrupadas.

Pregunta 11: ¿Cuál presenta menor distancia recorrida?

La estrategia de cercanía presenta la menor distancia en la mayoría de los escenarios: gana en básico (52 contra 76), cuello de botella (81 contra 99), despacho múltiple (97 contra 121), niveles z (68 contra 92) y

grande (2.481 contra 3.751). Empata en el escenario de cercanía (40 contra 40) y pierde en racks en esquinas (28 contra 26). Sumando los siete escenarios, la cercanía recorre 2.847 pasos contra 4.205 del orden, un 32,3 % menos.

En el escenario grande, la diferencia es de 1.270 pasos a favor de la cercanía (−33,9 %). Si se mira solo la generación de waves, `solo_cercania` es la que menos distancia total obtiene (2.167 pasos), seguida de `por_cercania` (2.481) y `por_pedido` (2.904). Es decir, la menor distancia global del experimento se logra combinando `solo_cercania` para armar waves y `cercania` para rutear.

La respuesta corta es: cercanía, con la salvedad de que no gana siempre. El contraejemplo de esquinas es parte de la respuesta, no una excepción que se pueda ignorar.

Pregunta 12: ¿Cuál presenta menor tiempo de ejecución?

En tiempo de pared, BFS suele ser más rápido que A* en nuestros escenarios. En el escenario grande, cercanía + BFS tarda 2,37 ms por wave contra 4,11 ms de cercanía + A*; orden + BFS tarda 4,71 ms contra 4,64 ms de orden + A*, una diferencia mínima. La razón es que el grafo no es lo bastante grande como para que el ahorro de nodos de A* compense el costo de mantener el heap y calcular la heurística en cada vecino. En un grafo mucho más grande, la relación podría invertirse.

Si se mira la cantidad de nodos expandidos, A* gana claramente: 9.262 contra 33.969 con cercanía y 18.117 contra 71.210 con orden. Esa métrica mide el trabajo del algoritmo, no el tiempo real. Para la consigna, las dos son útiles: los nodos expandidos explican por qué A* es una búsqueda informada, y el tiempo de pared muestra que en Python el costo constante importa.

La respuesta corta es: BFS para tiempo de ejecución en este prototipo; A* para cantidad de nodos expandidos. La estrategia de ruteo que menos tiempo total consume en el escenario grande es cercanía + BFS (33,22 ms para las 14 waves).

Pregunta 13: ¿Existe una estrategia que sea mejor en todos los escenarios?

No. El escenario de racks en esquinas es el contraejemplo más claro: `orden` recorre 26 pasos y `cercania` 28. La cercanía elige primero una parada que parece próxima, pero la geometría de las esquinas y los sentidos obligan a un rodeo; el orden, que no reordena, termina haciendo un recorrido más corto. En el escenario de cercanía hay empate, así que tampoco allí se puede decir que una sea estrictamente mejor.

Tampoco hay una estrategia mejor en todos los objetivos. `solo_cercania` gana en distancia, pero ignora la prioridad; `por_cercania` respeta la prioridad y queda en el medio; `por_pedido` conserva los pedidos completos y es más fácil de explicar. BFS gana en tiempo de pared, A* gana en nodos expandidos. La elección depende de qué se priorice y del layout del warehouse.

La conclusión es que conviene tener varias estrategias y compararlas. El sistema está pensado justamente para eso: la misma wave se puede correr con las cuatro combinaciones y ver las métricas. En un warehouse real, la estrategia se elegiría por layout y por objetivo operativo, no por una regla universal.

Pregunta 14: ¿Qué trade-offs encontraron entre tiempo de ejecución y calidad del recorrido?

- Distancia contra tiempo: la cercanía mejora la distancia, pero no siempre es más rápida de calcular. En el escenario grande, cercanía + BFS es a la vez más corta y más rápida que orden + BFS, pero en otros

escenarios el greedy puede hacer más búsquedas o recorrer más nodos. La mejora de distancia no sale gratis en todos los casos.

- Nodos expandidos contra tiempo de pared: A* expande mucho menos, pero en Python el heap y la heurística tienen costo. Para este tamaño de grafo, BFS suele ganar en tiempo; A* gana en la métrica de trabajo del algoritmo.
- Prioridad contra distancia: `solo_cercania` obtiene la mejor distancia, pero puede dejar pedidos prioritarios para el final. `por_cercania` pierde algo de distancia y conserva la prioridad. Es un trade-off operativo, no solo algorítmico.
- Dividir pedidos contra conservar pedidos completos: las estrategias por ítem agrupan mejor y reducen distancia, pero complican el seguimiento de un pedido y pueden generar más waves con ítems de varios pedidos. `por_pedido` es más simple de explicar y de auditar.
- Más pickers contra coordinación: usar varios pickers reduce el tiempo de preparación, pero agrega colisiones, esperas, desvíos y lógica de estacionamiento. En pasillos angostos, el beneficio puede desaparecer si los pickers se bloquean.
- Memoria contra recálculo: la tabla de distancias acelera el armado de waves, pero guarda una fila por rack. En el escenario grande son 126.276 entradas; en un warehouse mucho más grande habría que evaluar si conviene guardar todo o recalcular algunas filas.

Pregunta 15: ¿Qué modificaciones realizarían para utilizar el sistema en un Warehouse real?

El sistema actual es un prototipo académico. Para llevarlo a un warehouse real habría que agregar, como mínimo: integración con el WMS/ERP para recibir pedidos y stock en tiempo real; persistencia en una base de datos en lugar de archivos JSON; posiciones reales de los pickers (RFID, lectores de código o ubicación indoor); y un motor de re-planificación que reaccione a pedidos nuevos, productos faltantes, pickers caídos o zonas llenas.

También habría que modelar mejor el tiempo. Hoy la recogida es un tick base más un tick por altura; en un warehouse real influyen la distancia al producto, la altura, el peso, el envase, la experiencia del picker y el tiempo de escaneo. La simulación tendría que calibrarse con datos históricos y probablemente usar distribuciones en lugar de valores fijos. Lo mismo pasa con las colisiones: en un warehouse real hay pasillos más anchos, reglas de prioridad, zonas de espera y vehículos; el modelo de una celda por picker es una simplificación.

En escala, habría que pensar en un ruteo jerárquico o en precomputar distancias, en monitoreo y métricas en vivo, en seguridad (peso máximo, ergonomía, velocidad), en soporte para varios pisos y ascensores, y en una interfaz para supervisores. También haría falta validar el sistema con pruebas de aceptación y con usuarios reales, y definir qué hacer cuando el escenario es inviable: avisar, repartir la wave o cambiar la prioridad. Nada de esto está en el prototipo, y es importante decirlo: el objetivo del proyecto era entender los algoritmos, no construir un producto industrial.